

Hexaware-GenAI
Unlocking Innovation-
Your Path to AI-Driven Excellence

Scope Of the project

Project Title : Locator Finder Agent

Project Objectives:

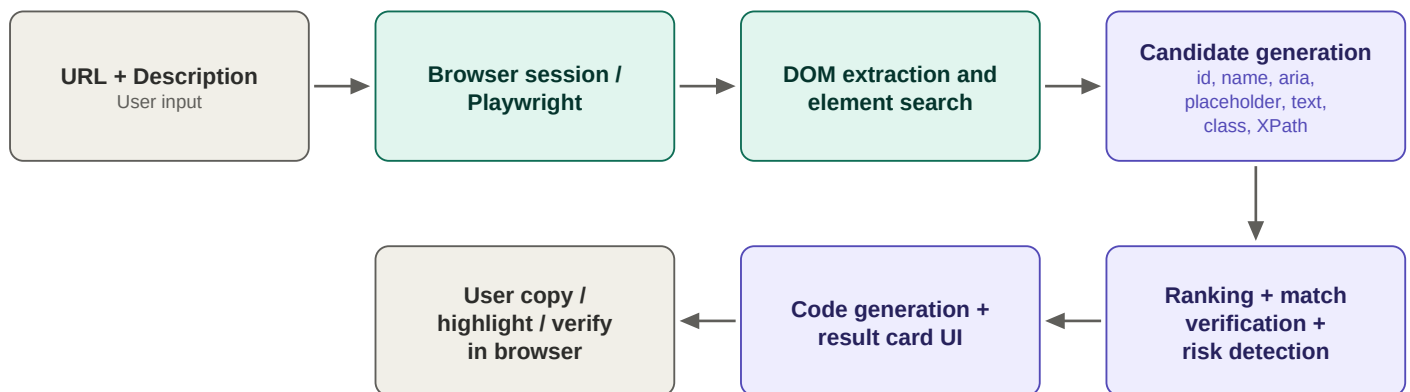
- Build a local web application that accepts a page URL and a natural-language element description.
- Use Playwright Chromium to open the page in a visible browser and inspect the live DOM.
- Identify the best Selenium locator for the target element using structural and attribute-based ranking rules.
- Return the best locator, ranked alternatives, match counts, risk flags, and ready-to-use code snippets.
- Support login flows by allowing the user to sign in manually in the browser before continuing.
- Detect frames, shadow DOM, and cross-origin layout conditions that affect selector reliability.
- Generate code in Python, Java, C#, JavaScript, and TypeScript for immediate reuse.
- Provide a local-first, session-aware interface that keeps browser state and element matches organized.

Deliverables:

- Local FastAPI backend with browser session management.
- Playwright-powered page inspection and live verification.
- DOM locator ranking and verification engine.
- Selenium code generator for multiple languages.
- Single-page HTML/CSS/JS frontend for interaction and result display.
- Test suite covering positive and negative selector scenarios.
- README, requirements, and setup instructions.

Design

Design Diagram:



Design Description:

Browser Interface: The application creates a visible local browser instance using Playwright Chromium. Users can open a URL, log in when needed, and continue the session without changing the browser state externally.

Locator Intelligence Module: The core engine inspects rendered DOM elements and extracts attributes such as id, name, data-testid, aria-label, placeholder, and classes. It ranks selectors according to their stability and realism, preferring real semantic attributes over generic text and positional XPath.

Verification and Safety Layer: The system verifies each candidate locator against the live page and reports match counts. It also flags conditions such as auto-generated IDs, ambiguous selectors, iframe usage, and shadow DOM differences so users can judge reliability before coding.

Result and Code Generation Layer: The UI displays the best locator, its match count, risk summary, and ranked alternatives. It also generates ready-to-use Selenium code in multiple programming languages so the user can paste directly into tests.

Session Management: Each browser interaction can be associated with a named session. The app supports creating, activating, refreshing, and closing sessions so users can work across multiple flows without stale page state.

Design

Workflow

- User enters a target URL in the local app.
- The app opens the URL in a visible Chromium browser.
- The user logs in manually when the site requires authentication.
- The user provides a natural-language description such as “username field” or “submit button”.
- The backend inspects the live DOM and collects candidate elements.
- The ranking engine prefers stable selectors such as ID, data-testid, and name-based attributes.
- Each candidate is validated live to compute match counts and capture risk information.
- The best locator is returned alongside alternatives, match counts, and reasoning.
- The app creates ready-to-use Selenium snippets in the requested language.
- The user can highlight the selected target in the browser or copy the snippet directly.

Test Cases

Positive Test Cases

- TC #1: Open a valid public page successfully.
- TC #2: Enter a clear description such as “login button” and get a valid locator result.
- TC #3: Submit a selector through the pick action and verify the match result is calculated.
- TC #4: Rank id and data-testid selectors above generic text or class-based alternatives.
- TC #5: Confirm the code generation output is valid for Python, Java, C#, JavaScript, and TypeScript.
- TC #6: Verify live highlighting selects the proper DOM element.
- TC #7: Validate session creation, activation, and closure workflow.

Negative Test Cases

- TC #8: Empty URL or missing description should return a validation error.
- TC #9: Invalid selector input should not produce a false-positive target.
- TC #10: Generic page header or nav links should not be ranked above the required form control.
- TC #11: Ambiguous selectors with multiple matches should be flagged with low stability.
- TC #12: Shadow DOM and iframe elements should be detected and flagged clearly.
- TC #13: Cross-origin frames should warn the user instead of pretending the selector is fully reliable.
- TC #14: Duplicate selector values should be deduplicated before ranking and display.

Tools and Code details

Third party tools Details:

Tool	Open source / Licensed	URL	Purpose
FastAPI	Open source	https://fastapi.tiangolo.com	Backend API server and local web app
Playwright	Open source	https://playwright.dev	Browser automation and DOM inspection
Pytest	Open source	https://pytest.org	Automated test execution
Python	Open source	https://www.python.org	Core backend language
HTML/CSS/JS	Open source	https://developer.mozilla.org	Front-end result rendering and interactions

Technologies used to develop in this project

Technology name	Version / usage	Purpose
Python	3.x	Application logic, API, and automation
FastAPI	Modern API framework	Local web server for browser and locator services
Playwright	Chromium automation	Live page inspection and DOM interaction
HTML / CSS / JavaScript	Front-end stack	Result card UI and interactivity
Pytest	Automated validation	Regression and feature testing

Functional Requirements

- Accept URL and description from the user.
- Open and maintain a local live browser session.
- Allow manual authentication when needed.
- Analyze DOM elements and rank candidates by stability.
- Return best locator plus ranked alternatives.
- Show risk notes such as low uniqueness or deep DOM context.
- Offer code generation for multiple language targets.
- Provide page highlighting and session management controls.

Non-functional Requirements

- Local execution only; no external backend dependency required.
- Fast response during page analysis.
- Clear and simple user interface.
- Reliable match verification for selector quality.
- Graceful handling of login pages, iframes, and shadow DOM.
- Maintainable and testable architecture.

Project Deliverables

- Browser-backed app for locator discovery.
- Final ranked selector output.
- Ready-to-use Selenium snippets.
- Manual and automated validation.
- Documentation and setup instructions.

Security and Scope Considerations

The application runs locally and does not upload page content to an external service. It relies on the browser session to inspect the page DOM in a real environment. Users are responsible for validating secure or restricted pages before automation.

Conclusion

The Locator Finder Agent is a practical local tool for discovering robust web element locators from real pages. It combines browser automation, live DOM analysis, match verification, selector ranking, and code generation in a single workflow. The project is especially useful for automation engineers, QA teams, and developers who need a faster and more reliable way to identify selectors for web testing and UI automation.

Thank you